

Inferno — Meeting Notes

Self-Hosted, Federated Chat Platform

Juan Enrique

February 2026

Contents

Executive Summary	3
The Problem	4
The Solution	4
What's Built Today	5
Messaging & Content	5
Voice & Video	5
Servers & Organization	5
Social	5
Federation (Cross-Instance)	5
Identity	5
Admin & Safety	6
Infrastructure	6
Architecture & Technical Moat	7
Single Instance	7
Horizontal Scaling	7
Federation Model	8
What Makes This Hard to Replicate	8
Trust, Compliance & Safety	9
CSAM Detection & Reporting	9
Federation Security	9
Data Protection	9
Trust Tiers & Rate Limiting	9
Revenue Model	10
Philosophy	10
Revenue Streams	10
Unit Economics	11

Distribution & Discovery	12
How Users Find Inferno	12
How Users Find Servers	12
Bootstrapping the Network	12
Growth Strategy & Roadmap	13
Success Metrics	13
Competitive Positioning	14
Technical Stack Summary	14
Why Now	16
Risks & Mitigation	16

Executive Summary

Inferno is a **blazing-fast, fully self-hosted chat platform** that delivers the user experience of Discord with the ownership guarantees of self-hosting and the portability of decentralized identity. Users sign up with email and password — behind the scenes, a Nostr cryptographic keypair gives them a **portable identity** that works across any Inferno instance, with no new accounts required.

What exists today: A working product — 573 RSpec specs passing, 39 ActiveRecord models, 30 granular role permissions, full voice and video via LiveKit, cross-instance federation operational. Text channels, DMs, reactions, threads, mentions, custom emoji, GIF search, file sharing, screen sharing, friend lists, blocking, profiles, online presence, typing indicators — all built and tested.

The pitch: Communities shouldn't be at the mercy of centralized platforms. Inferno gives them Discord-level UX with full ownership, data portability, and federated identity — free and open source under AGPL-3.0. Revenue comes from optional managed hosting (Inferno Cloud), cosmetic purchases, and eventually enterprise contracts.

Status: Working product in active development. Cross-instance federation operational. Seeking funding to launch the flagship instance, ship native apps, and build the hosted offering.

The Problem

Every online community today is a tenant on someone else's platform. Discord, Slack, and Teams control the infrastructure, the policies, and the kill switch.

What communities face:

- **Unilateral policy changes** — platforms rewrite terms of service without consent; communities must comply or leave
- **Identity verification mandates** — platforms force phone numbers, government ID, or payment info as conditions of use
- **Bans without recourse** — communities can be deleted overnight with no appeal and no data export
- **No data portability** — years of messages, files, and relationships locked inside a proprietary silo
- **No ownership** — the community's home exists at the pleasure of a corporation's trust & safety team

The result: communities live in a state of permanent precarity. One policy change, one moderation sweep, one acquisition — and everything disappears.

The Solution

Inferno is a full-featured chat platform that **looks and feels like Discord** but is **self-hosted, federated, and owned by the people who run it**.

Self-hosted: You run Inferno on your own server. Your data stays on your hardware. No third party can read your messages, change your rules, or shut you down.

Federated: Inferno instances connect to each other. A user on `gaming.chat` can join servers on `music.chat` without creating a new account. Friends, profiles, and identity travel across the network.

Invisible decentralization: Users sign up with email and password — the familiar flow. Behind the scenes, Inferno generates a Nostr cryptographic keypair that serves as their portable identity. Users never need to understand Nostr, manage keys, or choose relays. They just get an account that works everywhere.

Open source: AGPL-3.0 licensed. The code is public, auditable, and forkable. If the project disappears, the software survives.

What's Built Today

Messaging & Content

- **Text channels** organized by category with custom icons and permission overrides
- **Direct messages** — individual and group conversations with file sharing
- **Rich messaging** — reactions, custom emoji, custom stickers, GIF search (Tenor), link previews with embeds, @mentions, file attachments via Active Storage
- **Message management** — edit, delete, pin, bulk delete for moderators

Voice & Video

- **LiveKit SFU integration** — Discord-style voice channels with low-latency WebRTC
- **Screen sharing** with system audio support (Windows/Linux)
- **Moderation controls** — server mute, server deafen, move members between channels
- **Per-channel settings** — bitrate, user limits, video toggle

Servers & Organization

- **Categories** — group channels into collapsible sections
- **Roles** — 30 granular permissions (send messages, manage channels, kick/ban, voice controls, etc.)
- **Per-channel permission overrides** — fine-grained access control beyond server-wide roles
- **Invites** — shareable links with expiry, usage limits, and tracking
- **Server folders** — organize your server list on the sidebar

Social

- **Friend requests** — add users, including across instances
- **Blocking** — block users or entire remote instances
- **User profiles** — display name, bio, avatar, status, badges
- **Online presence** — see who's active in real time
- **Typing indicators** — see when someone is composing a message

Federation (Cross-Instance)

- **Cross-instance authentication** via Nostr NIP-42 challenge-response
- **Profile sync** — name, bio, avatar published to Nostr relays, pulled by remote instances
- **Remote users** — first-class citizens with roles, permissions, and moderation
- **Remote friends** — befriend users on other instances seamlessly
- **Shared channels** via Nostr NIP-29 relay groups
- **Federation controls** — open, allowlist, or closed federation modes

Identity

- **Nostr keypairs** generated automatically on signup
- **NIP-05 verification** — human-readable identifiers (`user@instance.chat`) verifiable across the web
- **Key export** — users can export private keys (nsec/ncryptsec via NIP-49) as a recovery mechanism

- **NIP-07 browser extension support** — power users can use nos2x, Alby, etc.

Admin & Safety

- **Instance configuration** — max users, servers, channels, roles, federation mode
- **Lockdown mode** — emergency pause on remote auth or local signups
- **Domain blocklists** — block specific instances from federating
- **User suspensions** — instance-wide or per-server, temporary or permanent
- **Legal holds** — preserve data for regulatory or legal requests; prevents pruning
- **Audit logs** — immutable logging of federation events, auth attempts, moderation actions
- **Data exports** — GDPR-compliant ZIP archive of user data
- **Message retention** — configurable time-based or storage-based pruning (pinned messages exempt)

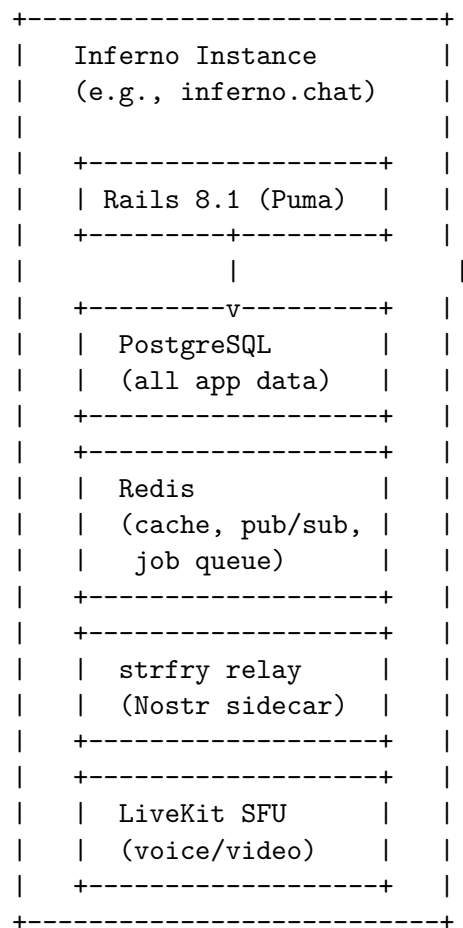
Infrastructure

- **Rails 8.1** on PostgreSQL, Redis, Sidekiq
- **Hotwire** (Turbo + Stimulus) + Tailwind CSS 4 — no separate frontend framework
- **ActionCable** for real-time WebSocket broadcasting
- **Active Storage** for S3/MinIO-compatible file uploads
- **strfry** Nostr relay sidecar for identity federation
- **573 RSpec specs**, CI/CD via GitHub Actions, Brakeman security linting
- **Docker Compose** production deployment, **Kamal** for zero-downtime deploys

Architecture & Technical Moat

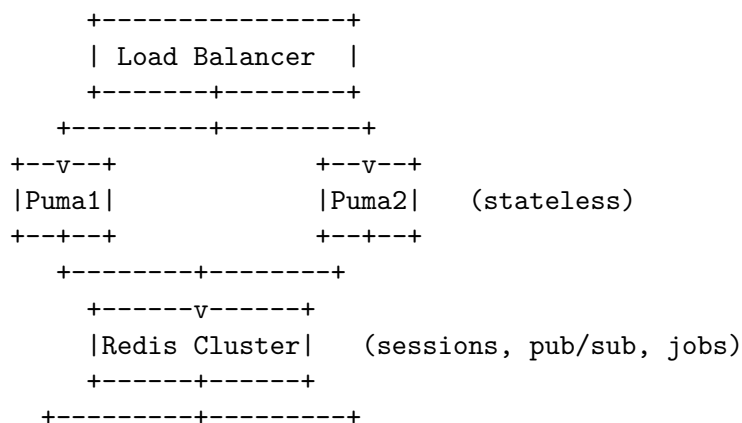
Single Instance

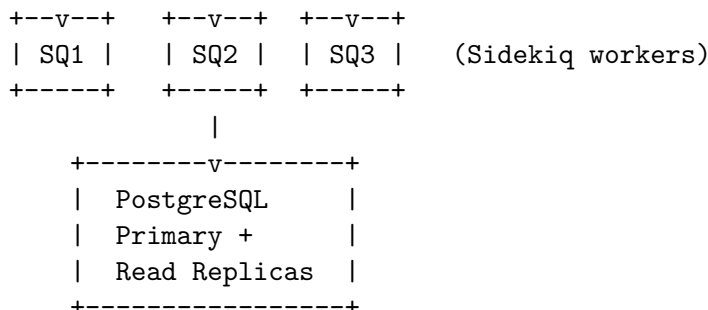
Each Inferno instance is a fully self-contained Rails monolith. No external API dependencies. Data stays local unless explicitly shared.



Horizontal Scaling

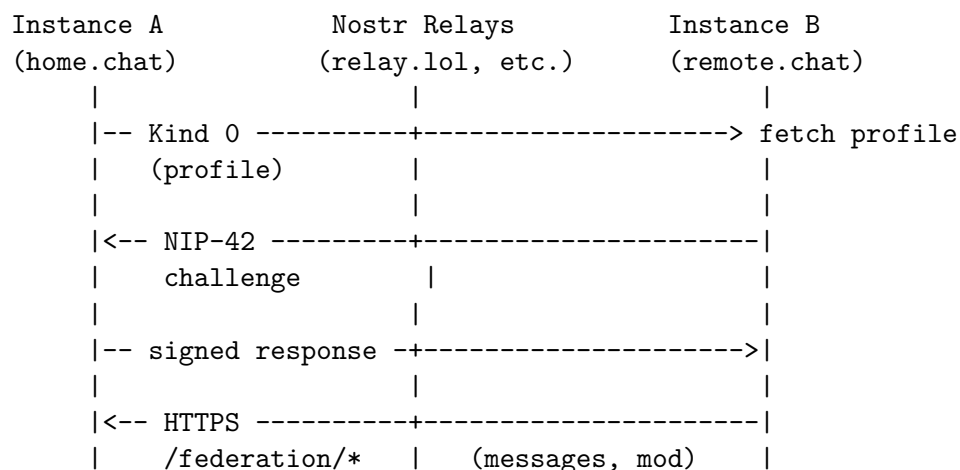
A single domain scales to thousands of concurrent users:





Federation Model

Instances communicate via Nostr relays (identity, profiles) and direct HTTPS (auth, moderation). Every federation event is cryptographically signed.



Cross-instance auth flow: User on Instance A clicks “Join” on a server hosted by Instance B. Instance B issues a challenge, Instance A signs it with the user’s Nostr key, Instance B verifies the signature and creates a local session. No new account. No password. Cryptographic proof of identity.

What Makes This Hard to Replicate

1. **Invisible decentralization** — email/password signup with Nostr identity generated behind the scenes; no other platform achieves this UX
2. **Hybrid Devise + Nostr identity** — local auth for speed, cryptographic identity for portability; deeply integrated at the model layer
3. **LiveKit + Ruby** — only SFU with a native Ruby SDK; no Node.js sidecar needed
4. **39-model domain** — channels, roles, permissions, federation, voice states, audit logs, legal holds — years of incremental domain modeling
5. **573 specs** — comprehensive test coverage that enables rapid iteration without regressions

Trust, Compliance & Safety

CSAM Detection & Reporting

Federal compliance (18 U.S.C. 2258A) built in from day one:

- **Scanning pipeline:** On file upload, compute SHA-256 + perceptual hash (dHash). Check against known-bad hash database (NCMEC, Project VIC). On match: auto-quarantine content, suspend user, create legal hold.
- **NCMEC reporting:** Auto-generate CyberTipline reports via API integration.
- **Federation broadcasting:** Suspension notices propagate to peer instances with configurable auto-action based on trust tier.

Federation Security

- **Domain blocklists** — reject all auth and communication from blocked instances
- **Allowlist/closed modes** — restrict federation to explicitly approved domains
- **Lockdown mode** — emergency kill switch for all new remote authentications
- **Audit trails** — every federation event immutably logged (auth attempts, token issuance, profile syncs, moderation actions)

Data Protection

- **GDPR data exports** — full user data as encrypted ZIP archive
- **Legal holds** — polymorphic holds on users, servers, or channels (prevents data pruning during legal proceedings)
- **Configurable retention** — time-based or storage-based message pruning with pinned-message exemptions

Trust Tiers & Rate Limiting

Instances and users are classified into tiers that control rate limits and federation access:

Tier	Description	Auth/min	Federation/min	API/min
0 — Blocklisted	All communication rejected	0	0	0
3 — Untrusted	New/unknown	5	10	30
2 — Standard	Established, verified NIP-05	15	60	120
1 — Verified	Payment verified, ToS signed	30	200	300

Tiers auto-promote based on history (successful auths, age, report count) and auto-demote on violations.

Revenue Model

Philosophy

All core features are free, forever. Messaging, voice, video, channels, roles, DMs, federation, identity, admin tools — no paywalls, no crippled free tier. Revenue comes from optional convenience and cosmetics.

Revenue Streams

1. Inferno Cloud — Managed Hosting

For users who love Inferno but don't want to run a server:

Tier	Price	Target
Self-Host	Free	Developers, power users
Starter	\$5/month	Friend groups, small communities
Community	\$15/month	Active communities, custom domain
Pro	\$40/month	Large communities, priority support

Margins: 70–80% after infrastructure costs. Self-hosted operators keep 100% of their revenue.

2. Cosmetics — Primary Revenue Driver

- Animated avatars, profile effects, custom colors, badges, premium sticker packs
- Stored as tags on user's Nostr profile (Kind 0) — **cosmetics travel across instances automatically**
- Remote instances render cosmetics by reading the profile; no payment processing needed on their end
- High margin, impulse purchases, non-essential

3. Server Boosts

- Users boost a server to show support (social currency)
- Boosted servers get a badge; boosters get a visible role
- Revenue goes to instance operator

4. Verification Badge

- Small one-time payment (Stripe or Lightning)
- Proof of personhood, higher trust tier, elevated rate limits
- Not required for any feature — signals legitimacy

5. Tipping

- User-to-user via Stripe or Bitcoin Lightning Zaps
- Instant settlement for Lightning; Nostr-native integration
- Platform takes a small fee on Cloud instances; self-hosted passes through 100%

6. Enterprise (Phase 8)

- SSO/SAML, SOC 2 compliance, SLAs, dedicated support
- \$500–\$5,000/month per customer

Unit Economics

Flagship instance at 500 DAU: Server cost ~\$50/month. Cosmetics ARPU $\$2 \times 500 = \$1,000$ /month. 95% gross margin. Cash-flow positive from month one.

At 50,000 DAU: Server cost \$500–\$2,000/month. Cosmetics + verification revenue ~\$125,000/month. 92% gross margin.

Distribution & Discovery

How Users Find Inferno

- **Reddit** — r/selfhosted (3M+), r/privacy (2M+), r/opensource — communities actively seeking Discord alternatives
- **Hacker News** — technical audience, high-signal launch surface
- **Nostr community** — built-in audience for Nostr-native software
- **Community migration** — target communities hit by platform policy changes (Discord TOS updates, bans, verification mandates)
- **Word of mouth** — one successful server migration creates a reference for others

How Users Find Servers

In-app Explore page querying Nostr relays for instance and server listing events — no central registry, no gatekeeper.

- **Instance discovery:** Instances publish metadata (name, domain, user count, federation mode) as Nostr events. Any instance can discover any other.
- **Server discovery:** Instances publish discoverable servers with tags and categories. Users browse and search across the entire federation network.
- **Cross-instance joining:** User clicks “Join” on a server from another instance. Nostr auth handles the rest — no new account needed.

Bootstrapping the Network

- **Seed list:** Curated list of known instances ships with the app; replaced by relay-based discovery as the network grows
- **Self-hosting growth:** Docker Compose guides, one-command setup, community-run instances (gaming, regional, interest-based)
- **Network effect moat:** Once users have cross-instance friends, shared channels, and portable identities, the federation network itself becomes the value. Leaving means leaving the network — but no single entity controls it.

Growth Strategy & Roadmap

Phase	Name	Description
1	Foundation (current)	Working product: messaging, voice/video, federation, 573 specs, AGPL-3.0
2	Flagship Instance	Launch <code>inferno.chat</code> , free open registration, migrate first community (50–200 people), prove product-market fit
3	Native Apps	Turbo Native (iOS/Android) + Tauri (desktop) + JSON API layer for decoupled clients
4	Discovery	Nostr relay-based server/instance discovery, in-app Explore page, self-hosting onboarding
5	Inferno Cloud	Managed hosting platform, one-click provisioning, custom domains, tiered pricing
6	Monetization	Cosmetics, boosts, verification, tipping via Stripe + Lightning Zaps
7	Product Depth	E2E encrypted DMs (NIP-44), 2FA, bots/integrations API, full-text search, threads, marketplace
8	Enterprise	SSO/SAML, SOC 2, ISO 27001, SLAs, dedicated support, on-premise consulting

Success Metrics

Year 1: Flagship instance with active community. 10,000 DAU. 50+ self-hosted instances. First Inferno Cloud beta customers. Native app beta.

Year 2: 100,000 DAU across the federation network. 1,000+ self-hosted instances. Inferno Cloud at 5–10% of user base. Enterprise pilots.

Year 3+: Profitability. Federation network effects. The default answer to “where should we host our community?” for anyone who cares about ownership.

Competitive Positioning

	Inferno	Discord	Matrix/Element	Revolt	Guided
Self-hosted	Yes	No	Yes	Partial	No
Federated	Yes (Nostr)	No	Yes (Matrix protocol)	No	No
Portable identity	Yes (Nostr keypair)	No	Partial (MXID)	No	No
Voice/video	Yes (LiveKit)	Yes	Partial	Yes	Yes
UX simplicity	Discord-like	Native	Complex	Discord-like	Discord-like
Onboarding	Email + password	Email + password	Choose homeserver, manage keys	Email + password	Email + password
Payments built-in	Cosmetics + Zaps	Nitro	No	No	No
License	AGPL-3.0	Proprietary	Apache 2.0	AGPL-3.0	Proprietary
Electron required	No	Yes	Yes	Yes	Yes
Data ownership	Full	None	Full	Partial	None

Key differentiators:

- Only platform combining Discord-level UX with native federation and portable identity
- Only self-hosted chat with built-in voice/video (LiveKit) that doesn't require Electron
- Invisible decentralization: users don't need to understand Nostr, choose relays, or manage keys
- Built-in monetization rails (Stripe + Lightning) that work across federated instances

Technical Stack Summary

Layer	Technology	Why
Backend	Rails 8.1 + PostgreSQL	Proven at scale, fast iteration, excellent ORM
Real-time	ActionCable + Redis	Native WebSocket support, multi-server pub/sub
Background jobs	Sidekiq + Redis	Battle-tested, reliable async processing

Layer	Technology	Why
Frontend	Hotwire (Turbo + Stimulus)	Server-rendered speed, no SPA complexity
Styling	Tailwind CSS 4	Utility-first, rapid UI development
Voice/Video	LiveKit (SFU)	Only SFU with Ruby SDK, Apache 2.0, 30MB idle
Identity	Nostr (secp256k1)	Decentralized, cryptographic, portable
Relay	strfry	High-performance C++ Nostr relay, minimal resources
Auth	Devise + Nostr NIP-42	Local speed + cross-instance cryptographic auth
Authorization	Pundit	Policy-based, clean separation of concerns
File storage	Active Storage	S3/MinIO compatible, framework-native
Testing	RSpec (573 specs)	Comprehensive coverage, CI/CD enforced
Security	Brakeman	Static analysis for Rails vulnerabilities
Deployment	Docker Compose / Kamal	Zero-downtime, reproducible, single-command
License	AGPL-3.0	Free forever, fork-safe

Why Now

- **Platform fatigue is real** — Discord policy changes, Twitter rebranding, gaming platform consolidation are driving communities to seek alternatives
- **Self-hosting surge** — the post-Twitter, post-Unity era has normalized community ownership; r/selfhosted has 3M+ subscribers
- **Nostr maturity** — 5+ years of development, 20+ NIPs, thousands of clients; the protocol is stable enough to build on
- **LiveKit availability** — enterprise-grade SFU now open source (Apache 2.0, 20k+ GitHub stars); makes voice/video viable for indie projects
- **Rails 8.1** — excellent for rapid iteration with built-in real-time, background jobs, and deployment tooling
- **Regulatory pressure** — GDPR, CCPA, DMA making centralized services expensive and complex; self-hosted solutions avoid most of this

Risks & Mitigation

Risk	Mitigation
Network effects favor incumbents	Invisible decentralization removes the UX barrier; portable identity reduces switching cost; federation means no single point of failure
Self-hosting is hard for non-technical users	Inferno Cloud captures those who love the product but want managed ops
Nostr adoption uncertainty	Nostr identity is invisible to end users; Inferno works standalone even if Nostr ecosystem contracts
Content moderation liability	Built-in compliance: CSAM detection, NCMEC reporting, audit logs, legal holds, federation suspension broadcasting
Scaling challenges	Proven stack (Rails + PG + Redis); LiveKit scales to thousands per node; horizontal scaling via read replicas and Sidekiq workers
Revenue uncertainty	Non-VC-dependent: flagship instance can cash-flow on a single \$40/month VPS; cosmetics revenue is gravy

Inferno is open source (AGPL-3.0) and in active development. For technical details, see the full documentation at the project repository.